

BANKING SYSTEM CLI

PROJECT REPORT

COURSE: Introduction To Problem Solving and Programming.

Introduction

This is a documentation for a Banking System CLI that I developed in Python for a course project. The system provides a user with the ability to create bank accounts, deposit and withdraw money, check balances, get account information after a successful captcha verification, and modify account information. All account data is saved to a local text file which ensures that the account information will be available in any subsequent execution of the program. The project grew gradually from a single file script which was debugged, restructured as a package with modules, and extended with input validation, a captcha-based lookup process and automated unit tests.

Problem Statement

Trying to keep track of bank account information manually without a system in place is prone to errors and will not work for more than a handful of accounts. This project addresses the

problem by developing a simple, structured banking system that can create accounts, deposit and withdraw safely and save account information between sessions. The program uses basic programming concepts: functions, data structures, file I/O, input validation and modular program design.

Functional Requirements

- Create an Account - Register a new account using an ADHAAR (12 digit) or PANCARD (alpha numeric) government identification and creating a unique 10-digit account number.
- Check Balance – lets you see the current balance of any active account by entering the account number.
- Deposit - put money into a current account and maintain the new balance.
- Extract - take money out of an account. If there is not enough money, refuse the transaction.
- Confirm a current name on an account before changing the name;
- Find Account - view account info (name, number, balance) protected by a randomly generated captcha to prevent casual or automated queries.
- Key in and confirm a correspondence to generate a transaction pin.

Non-Functional Requirements

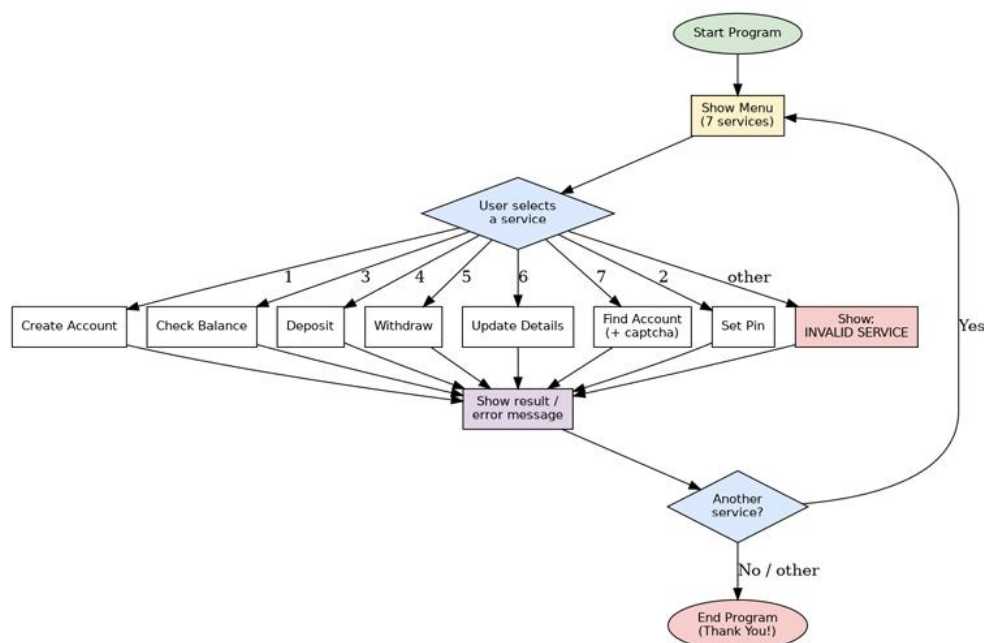
- User friendly - menu driven interface with clear prompts and specific error messages (e.g. INVALID ACCOUNT NUMBER, LOW BALANCE) for incorrect inputs or failed actions.
- Reliability – account information persists between program

executions (via a file); bad data in the file is skipped when reading rather than crashing the entire program.

- Maintainability - the logic is split into specialized modules (accounts, transactions, security, file storage, find account) rather than one large script, so each component can be altered or examined independently.
- Error handling - checking government id formats, account lookups, and withdrawals that are not funded adequately, rather than relying on the program not being able to report an error.

Process Flow / Workflow Diagram

Through workflow diagram we see the running sequence of the program created.



Storage

The project uses flat-file storage (Account.txt) rather than a relational database.

Design Decisions & Rationale

Flat-file storage over a database:

A database is more complex than flat-file storage.

As for a flat-file, each account is stored as a plain text entry, separated by blank lines. This makes it easy to read and parse with simple string manipulations we covered in class.

Re-reading the account file on every operation:

instead of storing account data in memory only,
each operation reads the current file contents

This allows using any account (not just the most recent one) with every feature

Captcha prior to revealing account details:

added specifically to the Find Account feature to prevent account details from being retrieved by simply guessing or iterating account numbers

Screenshots / Results

The following are the genuine output captured by the user, from creating an account to retrieving account details through the captcha gated find account feature

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 1
PLEASE ENTER YOUR NAME: xyz
Choose a Government ID of your choice (ADHAAR/PANCARD): ADHAAR
Enter your ADHAAR number: 789654123
ADHAAR entered
Your Account Number is: 2970369360
Do you want another service? (Yes/No): No
Thank You!
```

Creating account

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 2
Enter your account number: 2970369360
Enter Your six Digit Pin: 885500
Re enter Your six Digit Pin: 885500
Pin Created
Do you want another service? (Yes/No): No
Thank You!
```

Pin creation

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 3
Enter account number: 2970369360
account balance0
Do you want another service? (Yes/No): No
Thank You!
```

Balance check

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 4
Enter your account number: 2970369360
enter the Amount: 5000
5000 Has been credited,Avl Balance 5000
Do you want another service? (Yes/No): Yes
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 5
Enter your account number: 2970369360
Enter amount: 500
500 has been withdrawn. Avl balance4500
Do you want another service? (Yes/No): No
Thank You!
```

Deposit and withdraw

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 6
Do you want to Update your details?yes
Enter account number: 2970369360
enter your name to verify: xyz
What do you want to change?Name
Enter new name: abc
New name is abc
Do you want another service? (Yes/No): No
Thank You!
```

Updating Name

```
{'Create Account': 1, 'Pin': 2, 'Check Balance': 3, 'Deposit': 4, 'Withdraw': 5, 'Update': 6, 'Find Account': 7}
Enter The service that you require: 7
Enter your Account number: 2970369360
Captcha: AoIor
ENTER CAPTCHA AS SHOWN ABOVE: AoIor
Name:abc
Accountnum:2970369360
Balance:4500
Do you want another service? (Yes/No): No
Thank You!
```

Finding account

Testing Approach

Automated unit tests (tests/test_banking.py, using Python's built-in unittest framework)

test the core account operations, using a separate temporary account file for each test

to ensure tests don't interfere with each other. All 7 tests passed.

- Creating an account with a valid ADHAAR will create an account and store it correctly.
- Creating an account with an invalid ADHAAR will reject the creation.
- Deposit to account will add the correct amount to the balance.
- Withdraw from account with sufficient balance will reduce the balance.
- Withdraw from account with insufficient balance will reject the withdrawal.
- Creating two accounts separately will keep their balances separate
- Trying to check the balance of a non-existing account will return no results.

In addition to automated tests, the program was manually tested from start to finish

via the terminal interface (see Screenshots section) to ensure that the menu logic,

input validation, and multi-account handling all work together correctly.

Challenges Faced

1. Indentation errors: several early bugs came from code which looked correct, but had statements

out-of-place in their if/function block by incorrect indentation, which caused logic to be unconditional or never-run.

2. Global vs. local variable confusion: comparing a variable's name as a string ("Name") against

the variable itself (Name) caused several checks to silently fail, since the two aren't the same thing in Python.

Only the most recently created account was usable: early versions stored the current account in

3. single global variables, meaning operations were only possible against whichever was created last. Fixed by having every operation read in the full account list from file and look up the requested account by number.

4. A function which returned early inside a loop: a misplaced return inside a loop (rather than

after it) caused the account file to only ever be read as a single record, silently ignoring every account after the first.

Future Enhancements

- Store transaction pins permanently with the account record, and require that any pin be supplied before disclosing account details or making a transaction.
- Encrypt stored account details, rather than storing them in plaintext.
- Add a transaction log, which would store a record of every deposit or withdrawal ever made on any account, along with the time that it happened.
- Replace the flat-file storage with something more scalable like a proper database (SQLite would work for this).
- Provide a graphical user interface, perhaps using tkinter.

